Gen
AI

Module - 2

Runnables, Chains, Retrivers & RAG

-

muhammadosama.hq@gmail.com

Gen
AI

Topics to be Covered

- RunnableSequence
- RunnableParallel
- RunnablePassthrough
- RunnableLambda
- RunnableBranch
- LCEL
- LLM Grounding

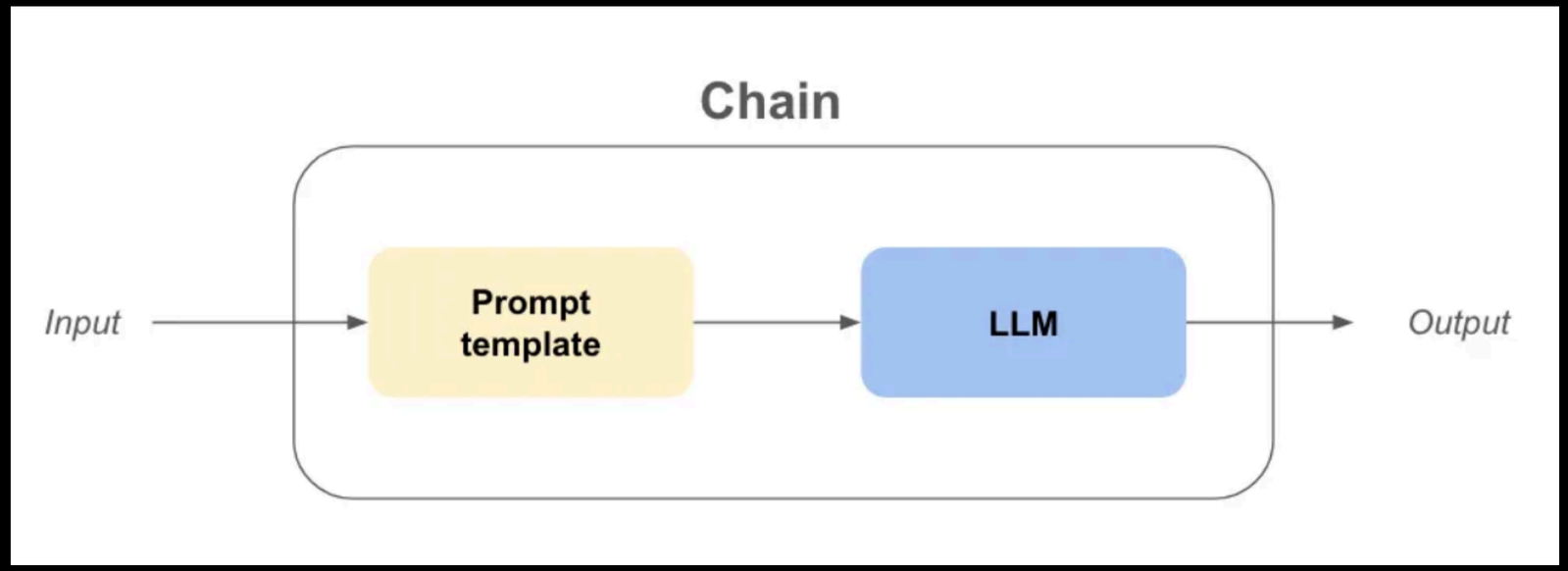
Gen
AI

Runnables & Chains

Gen
AI

What is Chain?

In LangChain, a chain is a sequence of calls.



What are Runnables?

In LangChain, a Runnable is the base interface for any component that can be executed (run) in a chain.

Runnable is a base interface,

If you're a Runnable, you must support at least .invoke(), .batch(), and .stream().

Gen
AI

Runnable Primitives

Most Usable runnable primitives are given below:

- RunnableSequence
- RunnableParallel
- RunnablePassthrough
- RunnableLambda
- RunnableBranch

Gen
AI

RunnableSequence

```python
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain.schema.runnable import RunnableSequence
from dotenv import load_dotenv

load_dotenv()

model = ChatOpenAI(model_name="gpt-4o-mini", temperature=0)
parser = StrOutputParser()

prompt = PromptTemplate(
    template="Translate the following sentence into Roman Urdu:\n\n{sentence}",
    input_variables=["sentence"]
)

chain = RunnableSequence(prompt, model, parser)
result = chain.invoke({"sentence": "I love learning about AI with LangChain."})
print(result)
```

Gen
AI

LCEL

-

What is LCEL?

LCEL stands for LangChain Expression Language.

It is a way to build GenAI pipelines in a declarative, composable, and chainable manner using operators like “|”

Gen
AI

Declarative Analogy

🧠 Simple analogy

Style	Example	Meaning
Imperative	"Go to the kitchen → Open the fridge → Take out milk → Pour into glass."	You describe *how* to do each step.
Declarative	"Bring me a glass of milk."	You describe *what* you want — system figures out *how*.

Example

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()

prompt = ChatPromptTemplate.from_template("Translate the following text into Arabic:\n{text}")
model = ChatOpenAI(model_name="gpt-4o-mini")

chain = prompt | model
response = chain.invoke({"text": "Good morning, how are you?"})
print(response.content)
```

Gen
AI

RunnableParallel

```python
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain.schema.runnable import RunnableParallel
from dotenv import load_dotenv
load_dotenv()

model = ChatOpenAI(model_name="gpt-4o-mini", temperature=0)
parser = StrOutputParser()
english_prompt = PromptTemplate.from_template("Translate to English: {text}")
french_prompt = PromptTemplate.from_template("Translate to French: {text}")

parallel_chain = RunnableParallel({
    "english": english_prompt | model | parser,
    "french": french_prompt | model | parser
})
result = parallel_chain.invoke({"text": "Je suis très heureux aujourd'hui."})
print(result)
```

Gen
AI

RunnablePassthrough

```
from langchain_core.runnables import RunnablePassthrough

passthrough = RunnablePassthrough()

result = passthrough.invoke({"message": "Hello, World!"})
print(result)
```

Gen
AI

RunnableLambda → part 1

```python
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain.schema.runnable import RunnableLambda
from dotenv import load_dotenv

load_dotenv()

parser = StrOutputParser()
model = ChatOpenAI(model_name="gpt-4o-mini", temperature=0)
prompt = PromptTemplate.from_template(
    "Generate a best speech on the topic {topic}"
)
```

Gen
AI

RunnableLambda → part 2

```
chain = prompt | model | RunnableLambda(
    lambda msg: {
        "json_result": parser.invoke(msg),
        "raw_result": msg
    }
)

result = chain.invoke({"topic": "AI"})
print(result)
```

Gen
AI

RunnableBranch → part 1

```python
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableBranch
from dotenv import load_dotenv

load_dotenv()
parser = StrOutputParser()
model = ChatOpenAI(model_name="gpt-4o-mini", temperature=0)

motivational_prompt = PromptTemplate.from_template(
    "Generate a motivational speech about {topic} with energy and passion."
)
informative_prompt = PromptTemplate.from_template(
    "Generate an informative speech about {topic} with facts and clarity."
)
```

Gen
AI

RunnableBranch → part 2

```
branch = RunnableBranch(
    (lambda x: x["topic"].lower() == "ai", motivational_prompt | model | parser),
    informative_prompt | model | parser  # default
)

result1 = branch.invoke({"topic": "AI"})
result2 = branch.invoke({"topic": "Healthcare"})

print("\n=== Motivational Speech (AI) ===\n", result1)
print("\n=== Informative Speech (Healthcare) ===\n", result2)
```

Task 1

Design a system that will take user input and return the response based on following conditions:

- If the query contains a question mark "?", sends it to question-answer chain.
- Otherwise, sends it to another summarization chain.

Task 2

A txt file with multiple emails are given summarize them differently based on topic sales, HR, finance

Follow Intructions:

- Classify topic (sales, HR, finance)
- Use different summarizer depending on topic
- print summary

Task 3

Input a Python code snippet, Explain what it does, find bugs, and suggest improvements.

Follow Intructions:

- Runnable 1: Explain code
- Runnable 2: Detect logical or syntax errors
- Runnable 3: Suggest improvements

RunnableParallel → Execute all together

Gen
AI

LLM Grounding

-

What is LLM grounding?

Anchoring an LLM's responses to real, external data so answers are factual, relevant, and reliable.

Why use it?

To reduce hallucinations, improve accuracy, and build user trust.

When to use it?

When answers must be up-to-date, verifiable, or based on private data.

Gen
AI

Exercise: Grounded LLM for History Assistant

An **AI-powered history assistant** for the general public.

The assistant answers historical questions using neutral, factual, and verifiable information from trusted public sources.

Mind Mapping of the Exercise

Step 1: Ungrounded Prompt

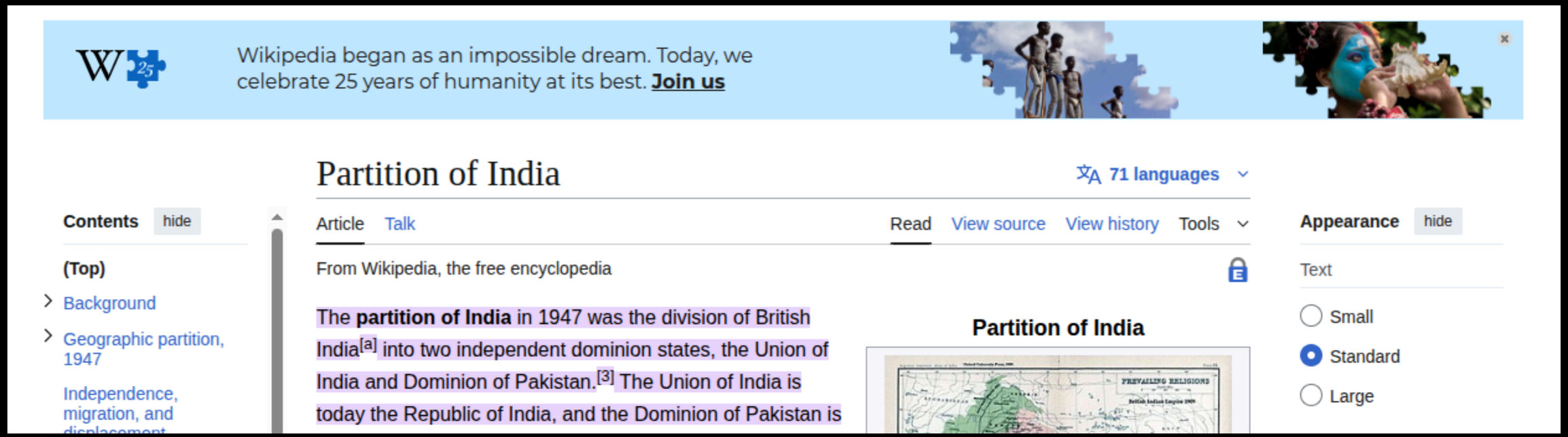
Ask ChatGPT:
"Explain the causes of the Indo-Pak partition."

and Observe Response

Gen
AI

Mind Mapping of the Exercise

Step 2: Grounded Prompt (This time ask with given article grounding)
"Explain the causes of the Indo-Pak partition."



Gen
AI

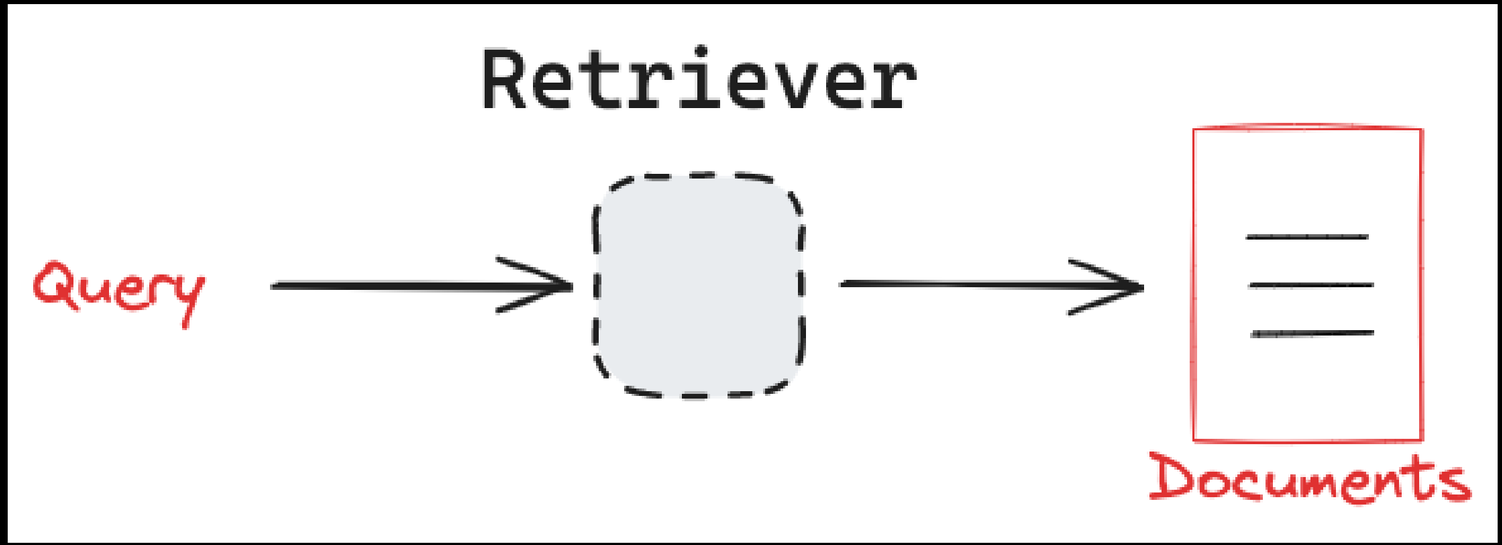
Retriver

-

Gen
AI

What is Retriver?

A Retriever is a component that fetches the most relevant documents of data in response to a user query so that an LLM can generate an accurate answer.



Gen
AI

Wikipedia Retriver → 1

```
from langchain_community.retrievers import WikipediaRetriever

# ----------------------------
# CONFIGURATION
# ----------------------------
retriever = WikipediaRetriever()

# ----------------------------
# QUERY
# ----------------------------
query = "why pakistan separated from india"
```

Gen
AI

Wikipedia Retriver → 2

```
# ---------------------------
# RETRIEVE DOCUMENTS
# ---------------------------
docs = retriever.invoke(query)

# ---------------------------
# ANALYSIS & VERIFICATION
# ---------------------------
print("\n================== RETRIEVAL SUMMARY ==================")
print(f"Total Documents (chunks) Returned: {len(docs)}")
```

Gen
AI

Wikipedia Retriver → 3

```
# Collect unique Wikipedia page titles
titles = {}
for doc in docs:
    title = doc.metadata.get("title", "Unknown Title")
    titles.setdefault(title, []).append(doc)

print(f"Unique Wikipedia Pages: {len(titles)}")
print("==================================================\n")
```

Gen
AI

Wikipedia Retriver → 4

```
# Print detailed breakdown
for title, chunks in titles.items():
    print(f"Wikipedia Page: {title}")
    print(f"Number of Chunks: {len(chunks)}")
    for i, chunk in enumerate(chunks, start=1):
        print(f"  Chunk {i} length: {len(chunk.page_content)} characters")
    print("-" * 50)
```

RAG

-

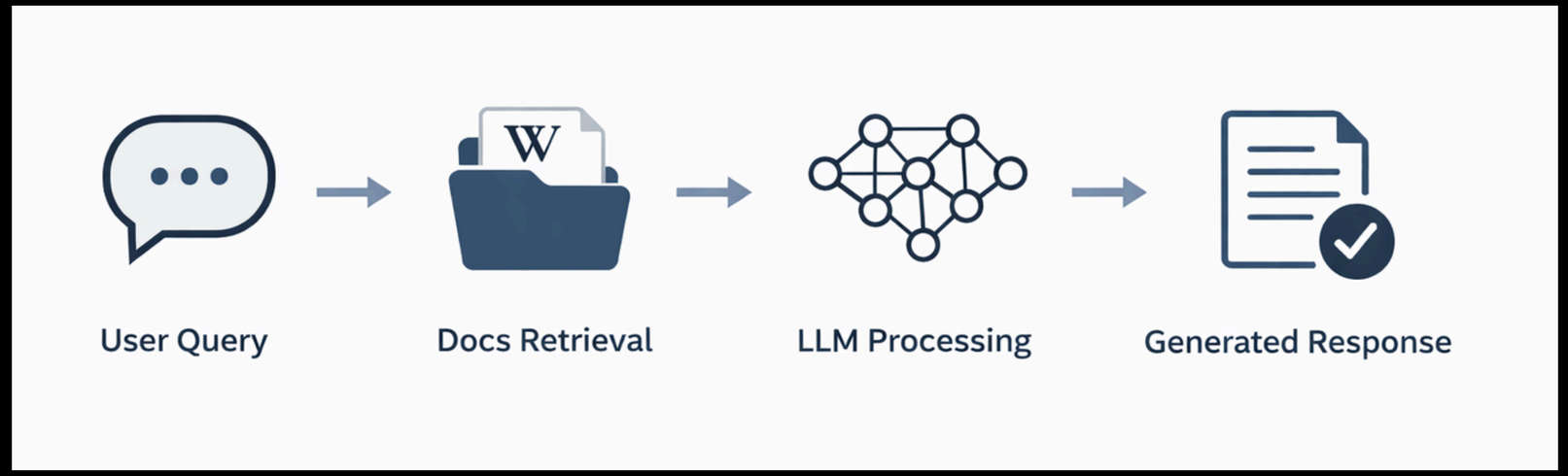
Gen
AI

RAG

Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with large language models.

Key Benefits

- More accurate and up-to-date answers
- Reduced hallucinations
- Easier to verify and cite sources



Gen
AI

Use Case: AI History Assistant

An **AI-powered history assistant** for the general public.

The assistant answers historical questions using wikipedia as verified public source

Gen
AI

Code Walkthrough → 1

```
from langchain_community.retrievers import WikipediaRetriever
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Set up retriever for Wikipedia (loads top 3 pages by default)
retriever = WikipediaRetriever()
```

Gen
AI

Code Walkthrough → 2

```
# RAG prompt template
prompt = ChatPromptTemplate.from_template(
    """You are a history assistant for the general public.

You will receive:
1) A user question
2) Wikipedia passages retrieved by the system

Your job:
- Answer the question using ONLY the provided Wikipedia passages.
- Do NOT add any outside facts, opinions, or interpretation.
- If the answer is not fully supported by the passages, say:
  "I cannot answer that based on the provided sources."
```

Gen
AI

Code Walkthrough → 3

```
Example 1:
User question: What were the main causes of the Indo-Pak partition?
Wikipedia passages: [PASSAGES HERE]

Answer: [Use ONLY the provided passages]

Example 2:
User question: What caused the French Revolution?
Wikipedia passages: [PASSAGES HERE]

Answer: [Use ONLY the provided passages]
```

Gen
AI

Code Walkthrough → 4

```
Example 3:
User question: What were the causes of World War II?
Wikipedia passages: [PASSAGES HERE]

Answer: [Use ONLY the provided passages]

Now answer:
User question: {user_question}
Wikipedia passages: {retrieved_passages}"""
)
```

Gen
AI

Code Walkthrough → 5

```
# LLM for generation (use your preferred model/provider)
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# Assemble RAG chain: retrieve → format → prompt → LLM → parse
rag_chain = (
    retriever
    | (lambda docs: "\n\n".join(doc.page_content for doc in docs))  # Join docs
    | prompt
    | llm
    | StrOutputParser()
)
```

Gen
AI

Code Walkthrough → 6

```
# Run the history assistant
question = "What were the key events of the Battle of Thermopylae?"
response = rag_chain.invoke({"question": question})
print(response)
```

Gen
AI

End of Class

-